

Convolutional Neural Networks

Objectives

- Understanding

convolutions • CNN building

blocks

- Example CNN

Convolutions

- Sounds fancy but it is not!
- Every time we do image blurring, smoothing, sharpening, edge detection, etc. we are doing convolutions
- Convolutions are one of the most critical, fundamental building blocks in computer vision and image processing.

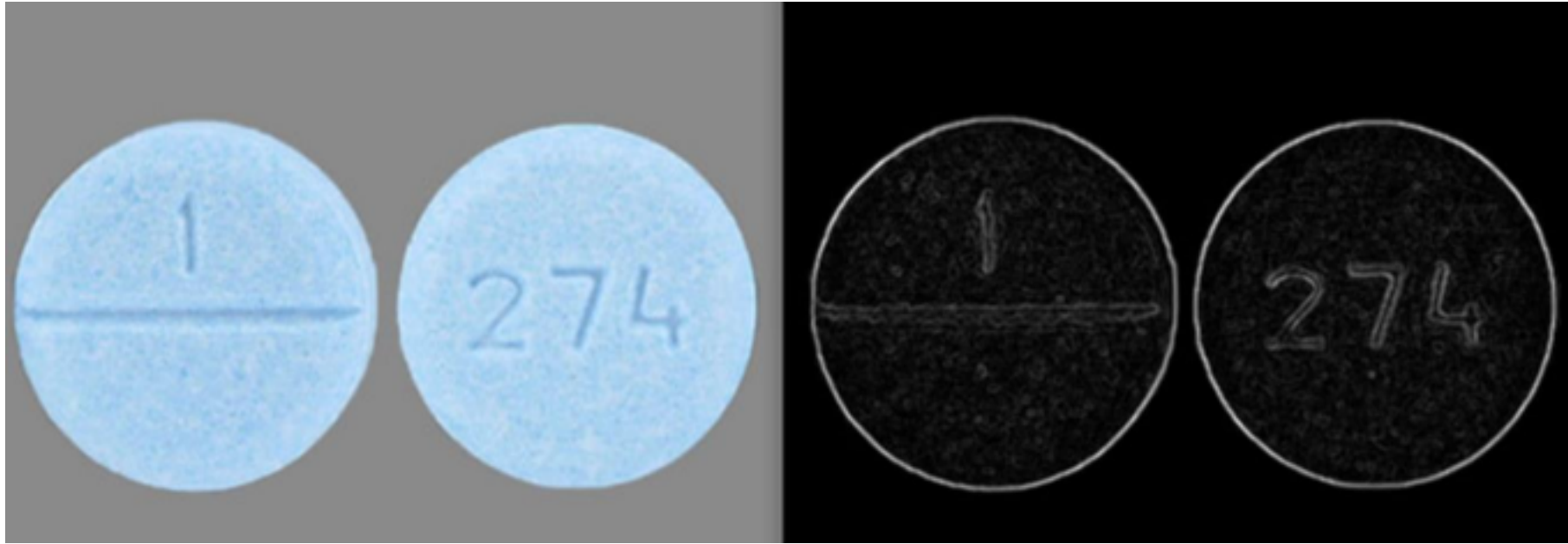
What is a convolution?

“In terms of deep learning, an (image) convolution is an element-wise multiplication of two matrices followed by a sum”

1. Take two matrices (which both have the same dimensions).
2. Multiply them, element-by-element (i.e., not the dot product, just a simple multiplication).
3. Sum the elements together.

Edge Detection Example

- Horizontal Edges
- Vertical Edges



So how do we detect edges?

- Vertical Edge Detection in a Grey Scale

3	0	1	2	7	4
1	5	8	9	3	1

Image

*

Convolution

Filter or Kernel

3 x 3

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Image

1	0	-1
1	0	-1
1	0	-1

*

Convolution

Filter or Kernel

3 x 3

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

1	0	-1
1	0	-1
1	0	-1

5	6	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6 x 6

Convolution

4 x 4

Filter or Kernel

3 x 3 * =

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

☀			

3 ¹	0 ⁰	1 ⁻¹	2	7	4
1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Convolution

4 x 4

1	0	-1
1	0	-1
1	0	-1

Filter or Kernel

3 x 3 * =

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

-5			

1	0	-1
1	0	-1
1	0	-1

1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Convolution

$\ast$ =
3 x 3

4 x 4

Filter or Kernel

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

-5	☀		



1	0	-1
1	0	-1
1	0	-1

Convolution

$*$
3 x 3



4 x 4

Filter or Kernel

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

-5	-4		

1	0	-1
1	0	-1
1	0	-1



Convolution



4 x 4

Filter or Kernel

6 x 6

So how do we detect edges?

- Vertical Edge Detection in a Grey Scale Image

-5	-4	0	8
-10	-2	2	3

0	-2	-4	-7
-3	-2	-3	-16

1	0	-1
1	0	-1
1	0	-1



3 x 3

Convolution

4 x 4

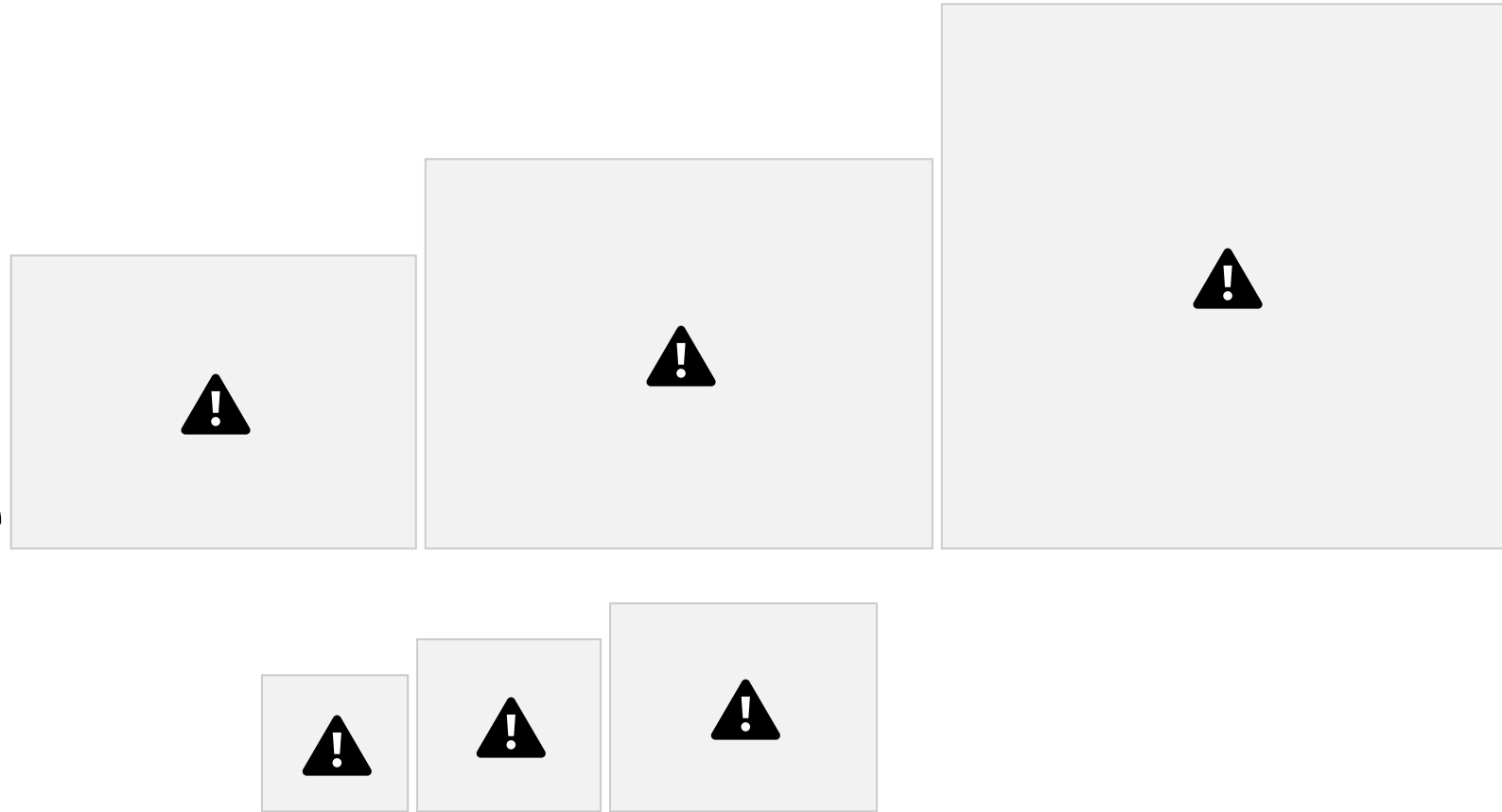
Filter or Kernel



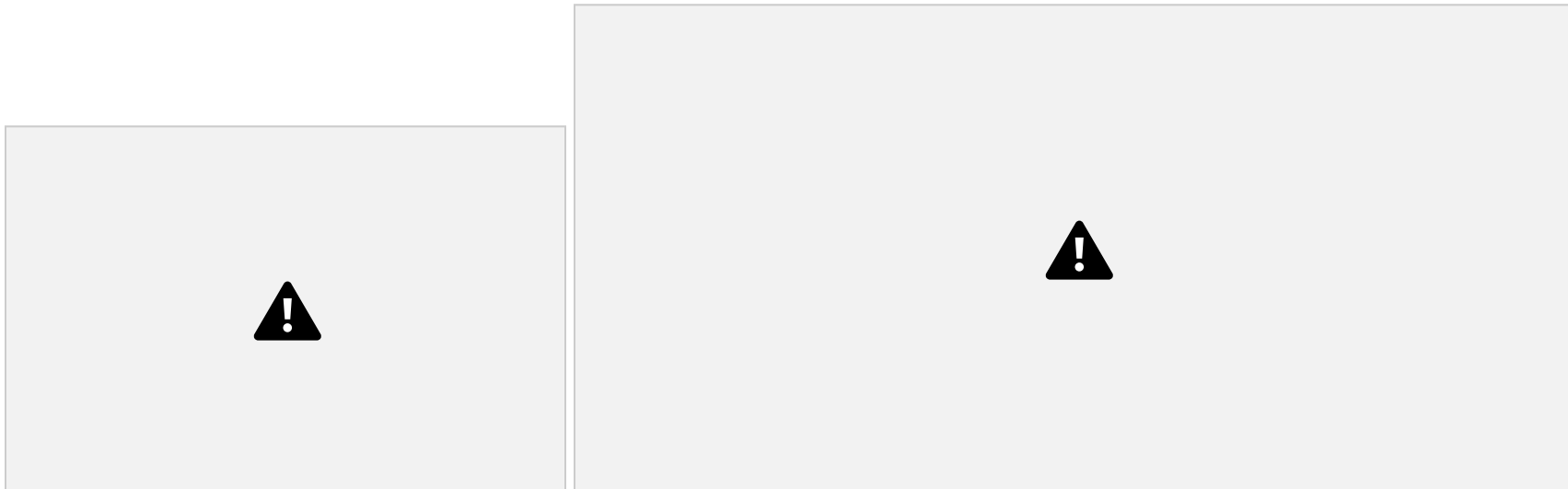
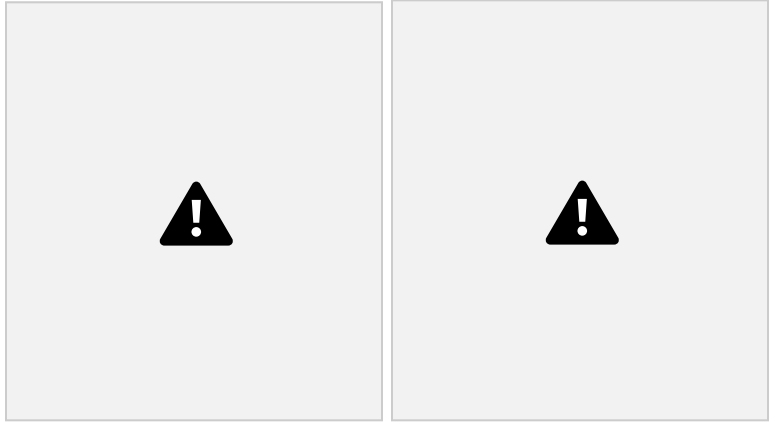
6 x 6

Vertical Edges: Another Simplified

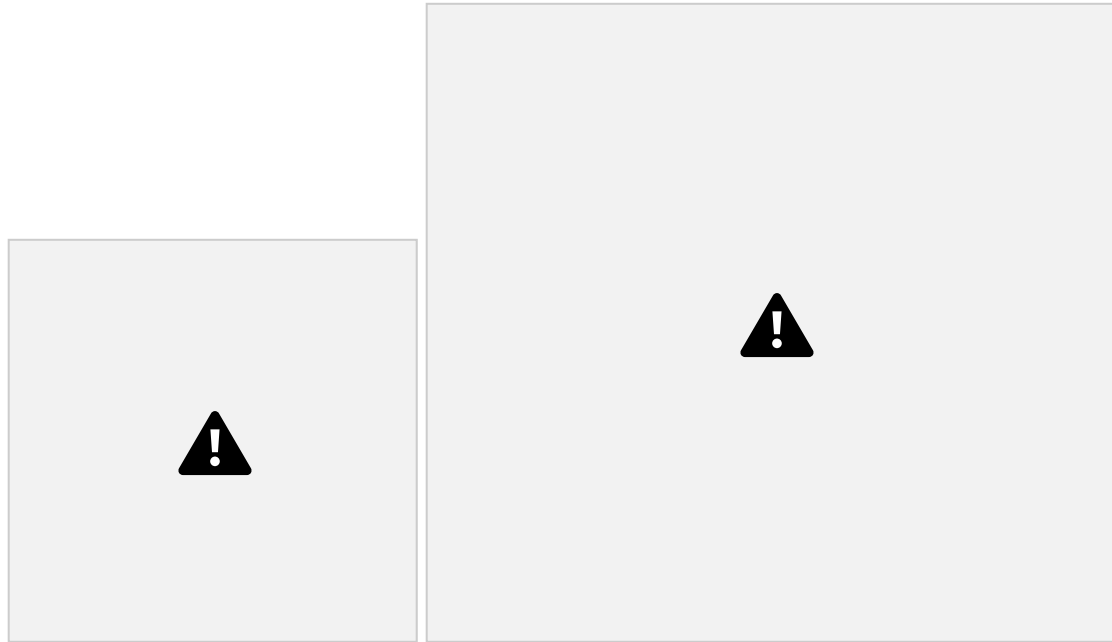
Example



Vertical and Horizontal Edge Detection



Learning Filters



Convolutional Neural Networks
(CNNs)

Padding



--	--	--

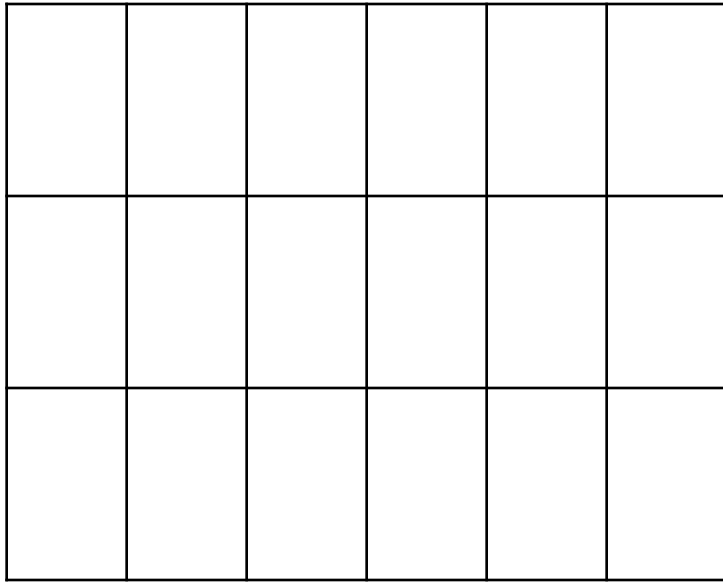
6 x 6

Padding

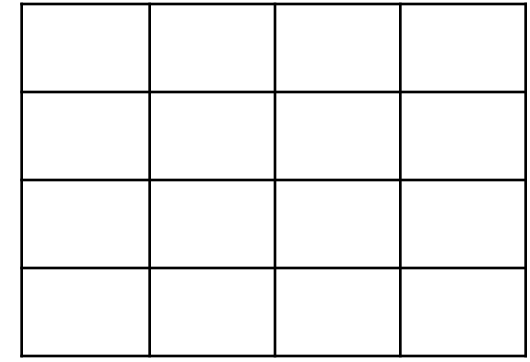
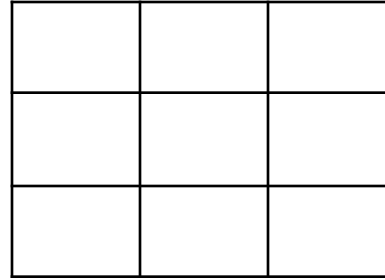
--	--	--	--	--	--

--	--	--	--	--	--	--	--	--	--	--	--

3 x 3 4 x 4



6 x 6



3 x 3 4 x 4

$$n \times n \quad f \times f \quad (n - f + 1) \times (n - f + 1)$$

Padding

1. Thus, the size decreases (shrinking output)

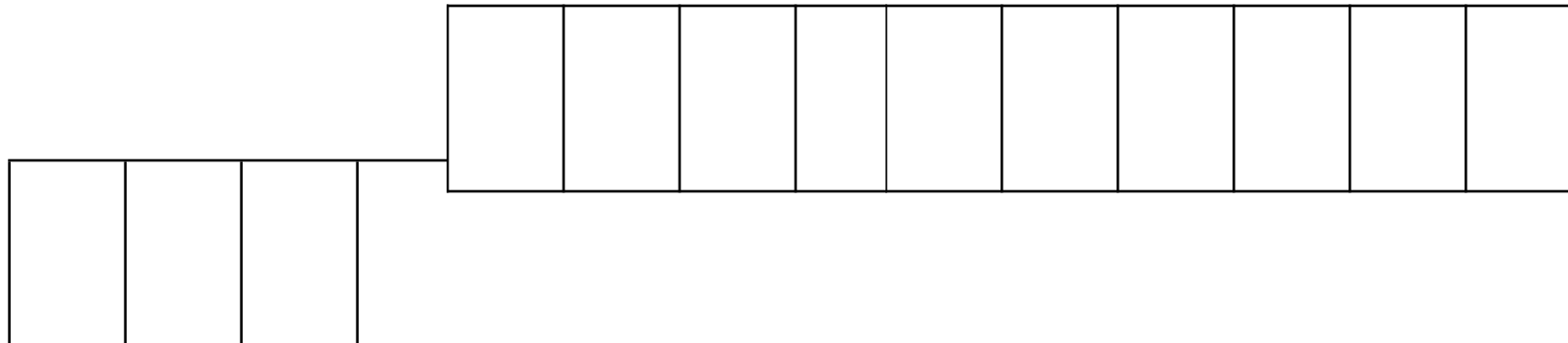
2. As well as, the pixels on the edges are used less than those in the center of the image

- And in order to fix these problems, we can do **padding** (pad the image with additional border of pixels)

Padding



What will be the dimensions of the output if we did padding?



6 x 6

3 x 3



--	--	--

New Dimensions After Padding

0	0	0	0	0	0
0					
0					
0					

0					
0					
0					
0	0	0	0	0	



8 x 8

--	--	--

3 x 3

New Dimensions After Padding

0	0	0	0	0		0	0
---	---	---	---	---	--	---	---

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0							0

8 x 8

3 x 3

6 x 6

Dimensions with Padding

- Input Size:
- Filter Size:
- Padding:
- Output Size:

Padding (Dimensions)

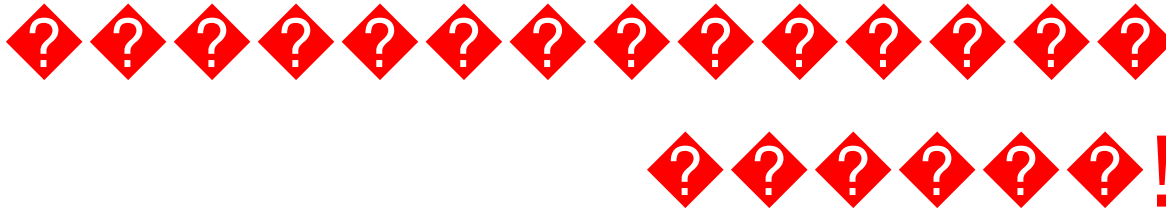
- Input Size: $n \times n$
- Filter Size: $f \times f$
- Padding: p
- Output Size: $(n + 2p - f + 1) \times (n + 2p - f + 1)$

Padding (Terminologies)

- If no padding: “Valid Convolutions”
- If padding to keep the output size the same as input size: “Same Convolutions”

$$\diamond \diamond = \frac{\diamond \diamond - 1}{2}$$





Stride



moving filter across the input, we stopped at each coordinate

Stride



What if we used a stride of 2?

Dimensions with Stride

- Input Size: $n \times n$
- Filter Size: $f \times f$
- Padding: p

- Stride: S
- Output Size: $\frac{H_1 - H_2 + 1}{S} \times \frac{W_1 - W_2 + 1}{S} \times D + 1$

Convolutions over a Volume

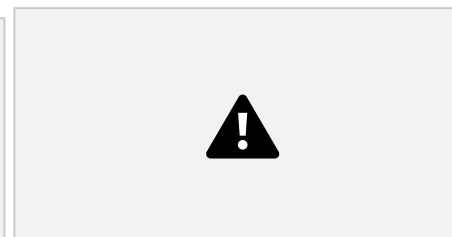
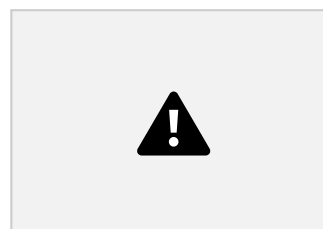
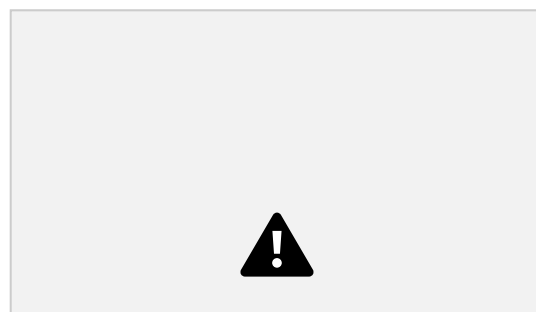
Convolutions on RGB Image



$3 \times 3 \times 3 \times 4 \times 4$

$6 \times 6 \times 3$

Convolutions on RGB Image



3 x 3 x 3 4 x 4

6 x 6 x 3

Multiple Filters



Vertical Edges

Horizontal Edges

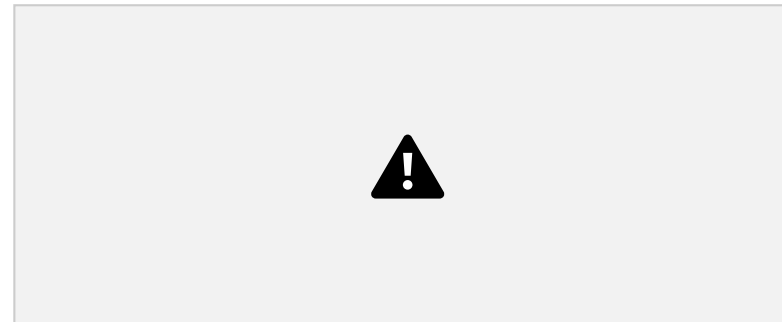


Multiple Filters



Vertical Edges

Horizontal Edges



Activation Maps – combined as a volume

Activation Maps



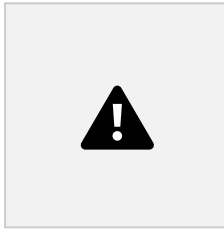
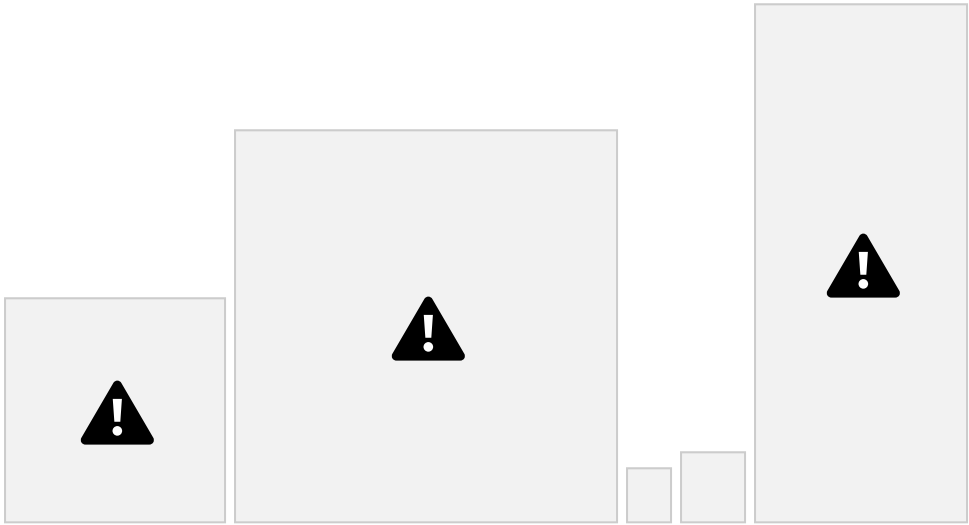
Activation Maps



After obtaining the K activation maps, they are stacked together to form the input volume to the next layer in the network

One Layer of a CNN

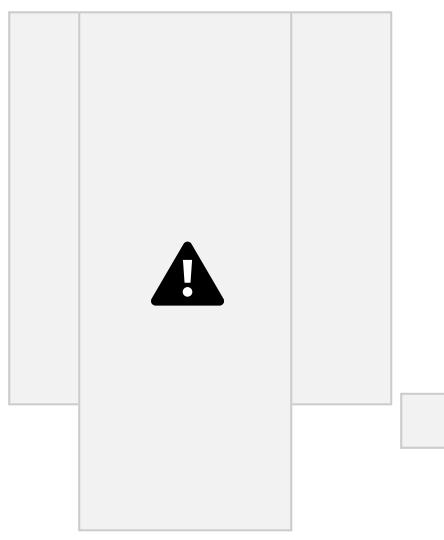




One Layer of a CNN



One Layer of a CNN

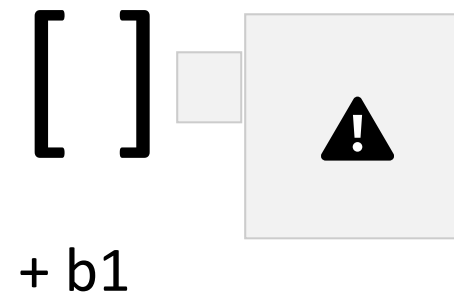


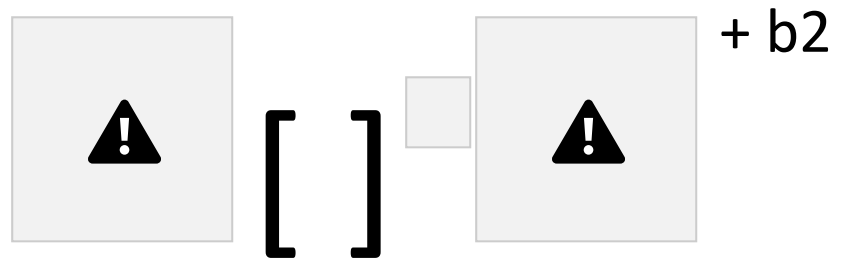
$$\begin{bmatrix} \end{bmatrix} + b_1$$

$$\begin{bmatrix} \end{bmatrix} + b_2$$

 ReLU   ReLU

One Layer of a CNN

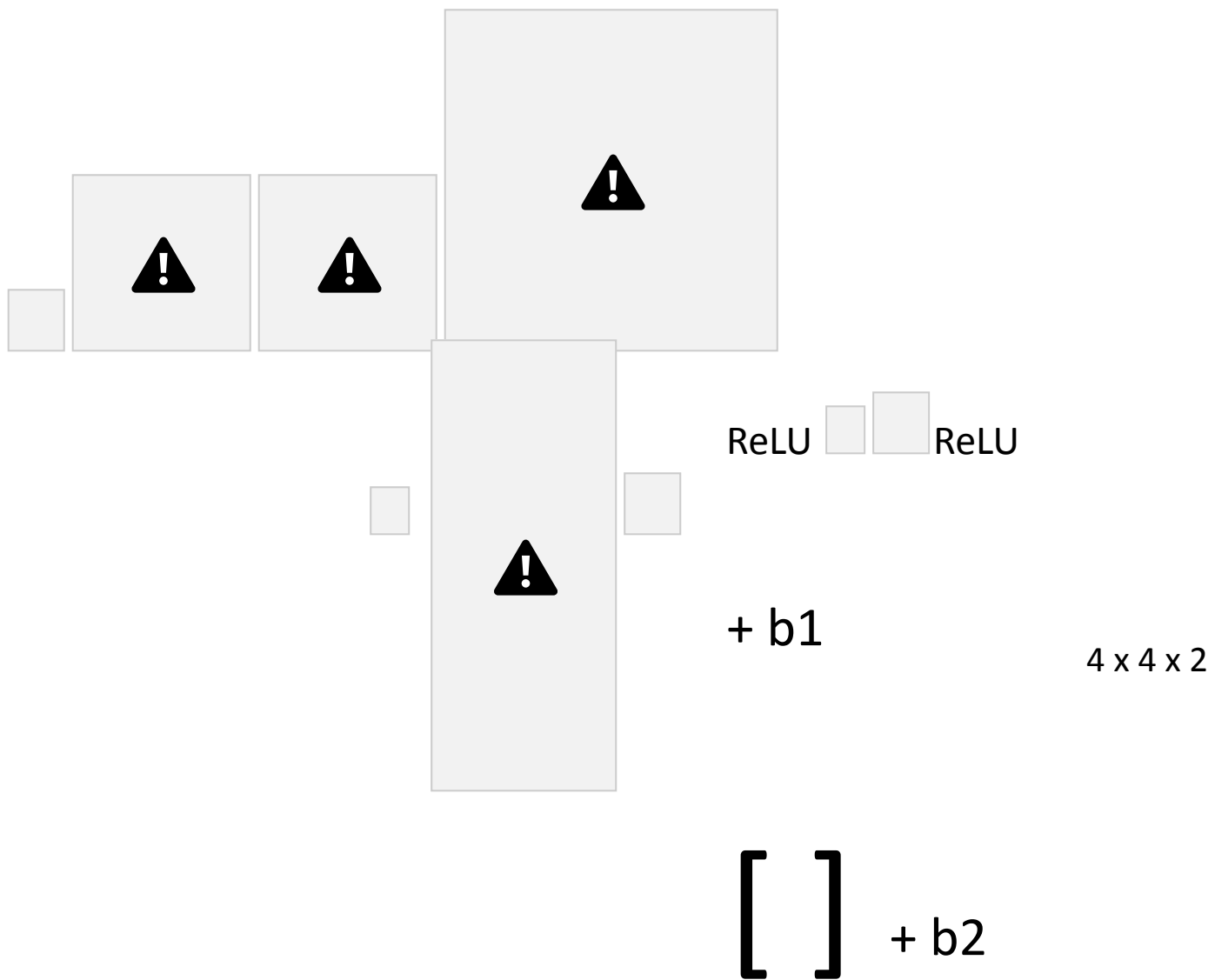




One Layer of a CNN



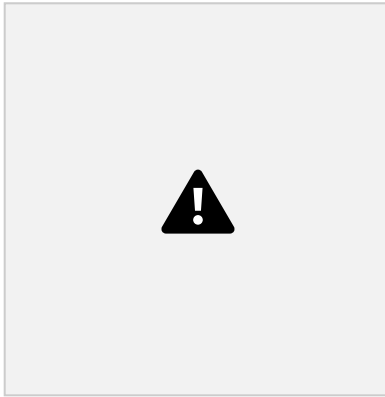
$[]$

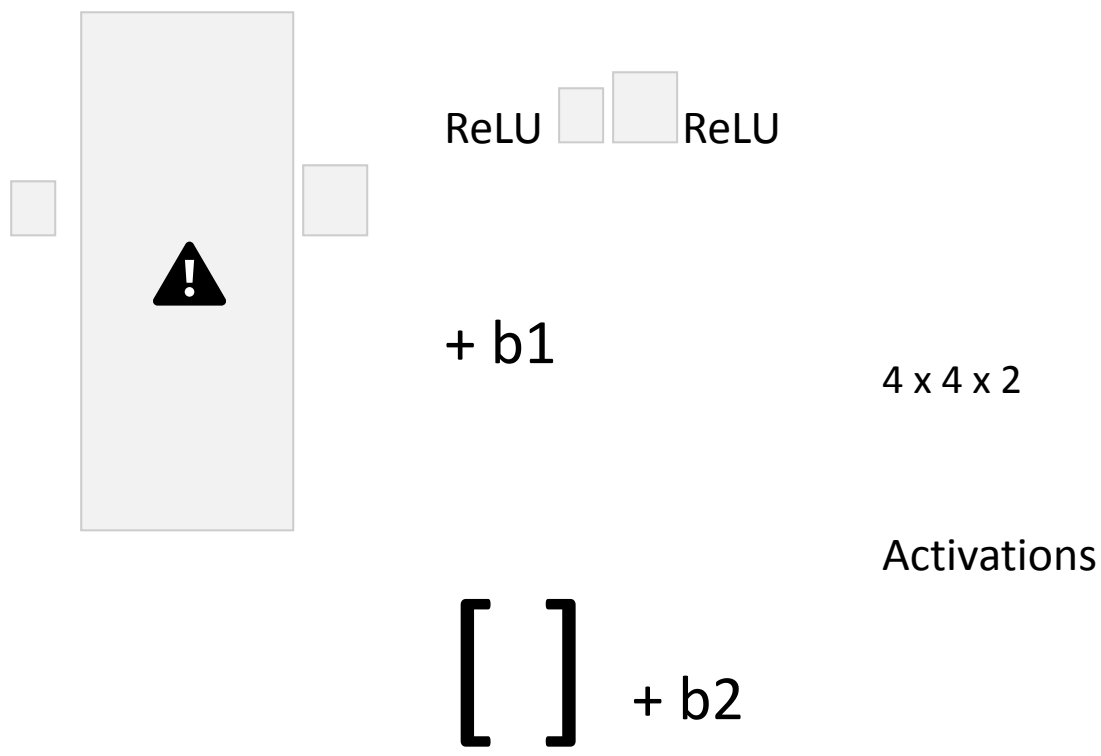


One Layer of a CNN



[]

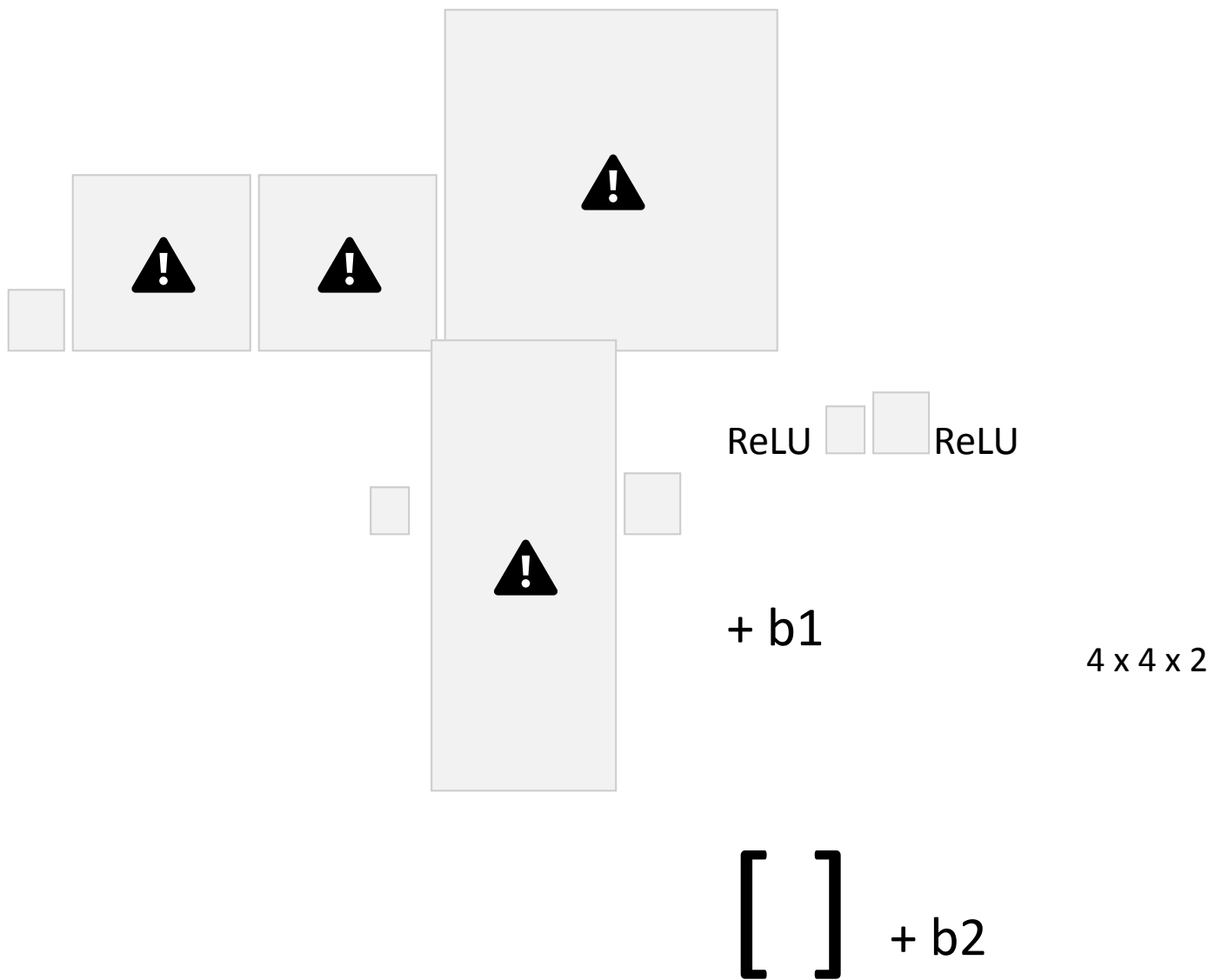




One Layer of a CNN



[]



ReLU is just for example, you can apply any non-linearity

Number of Parameters in a

Layer • Let's say that in a layer:

- You have 10 filters
- Each filter is of size $3 \times 3 \times 3$
- How many parameters does this layer have?

Number of Parameters in a

Layer • Let's say that in a layer:

- You have 10 filters
 - Each filter is of size $3 \times 3 \times 3$
 - How many parameters does this layer have?
-
- For each filter we have $27 + 1$ parameters (1 for bias)
 - Total number of parameters for 10 filters = 280

Types of Layers in a Convolutional

Net: • Convolutional Layers (CONV)

- Pooling Layers (POOL)
- Fully connected (FC)

Pooling

- In convolutional neural nets, the input size decreases as we move deeper in to the network

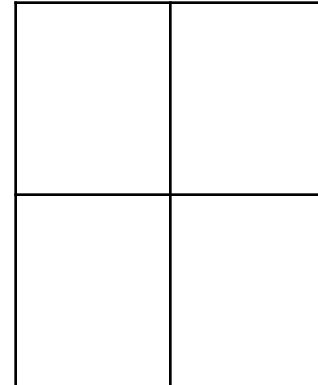
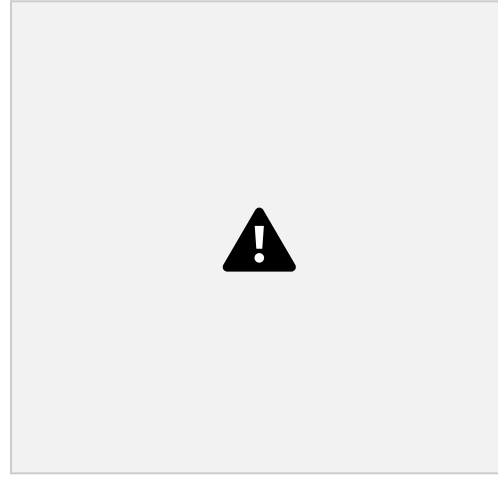
- One method to reduce the input size is _____

Pooling

- In convolutional neural nets, the input size decreases as we move deeper in to the network
- One method to reduce the input size is stride
- Another way to do this is via **pooling**

Pooling Layer:

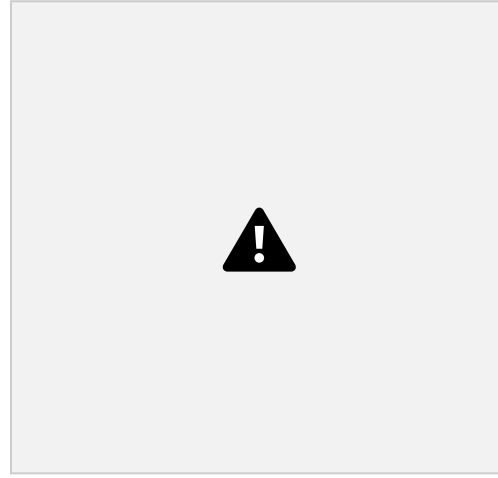
Max Pooling



Apply a max pooling filter of size 2 x 2 with a stride of 2

Pooling Layer:

Max Pooling



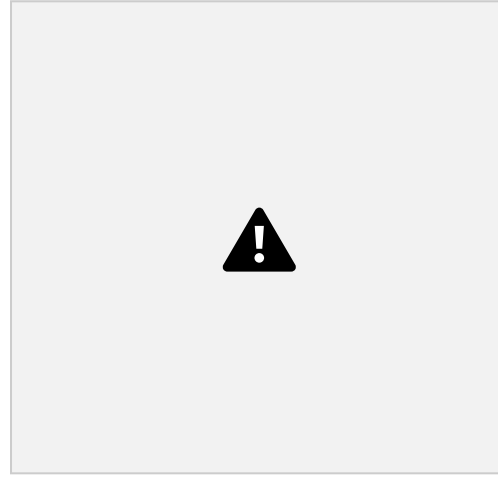
9	2
6	3

- Filter size and stride length are the hyperparameters of the pooling layer •

There are no parameters to learn

Pooling Layer:

Max Pooling



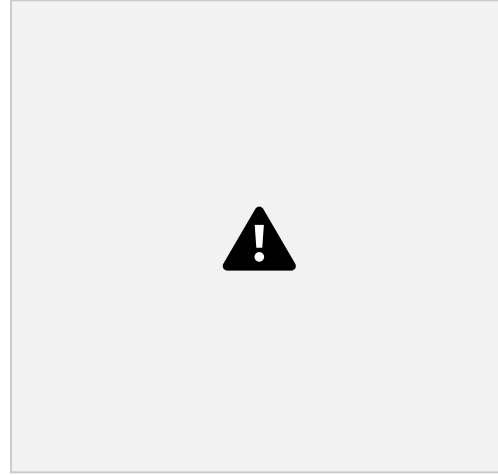
9	2
6	3

- Max pooling can be thought of as a filter that works on a feature map •

Looking for a particular feature

- Preserving the strength of finding that feature in a region

Pooling Layer:



Max Pooling

9	2
6	3

- Finally, you do pooling on each channel in the input separately

Example

9

f: 5

s: 1

6 filters

32 x 32 x 3

Example

CONV1

9

f: 5

s: 1

6 filters

Max Pooling f: 2

s: 2

32 x 32 x 3 28 x 28 x 6

Example

CONV1

POOL1

9

f: 5

s: 1

6 filters

Max Pooling

f: 2

s: 2

f: 5

s: 1

16 filters

28 x 28 x 6

32 x 32 x 3

14 x 14 x 6

Example

f: 5

s: 1

6 filters

Max Pooling

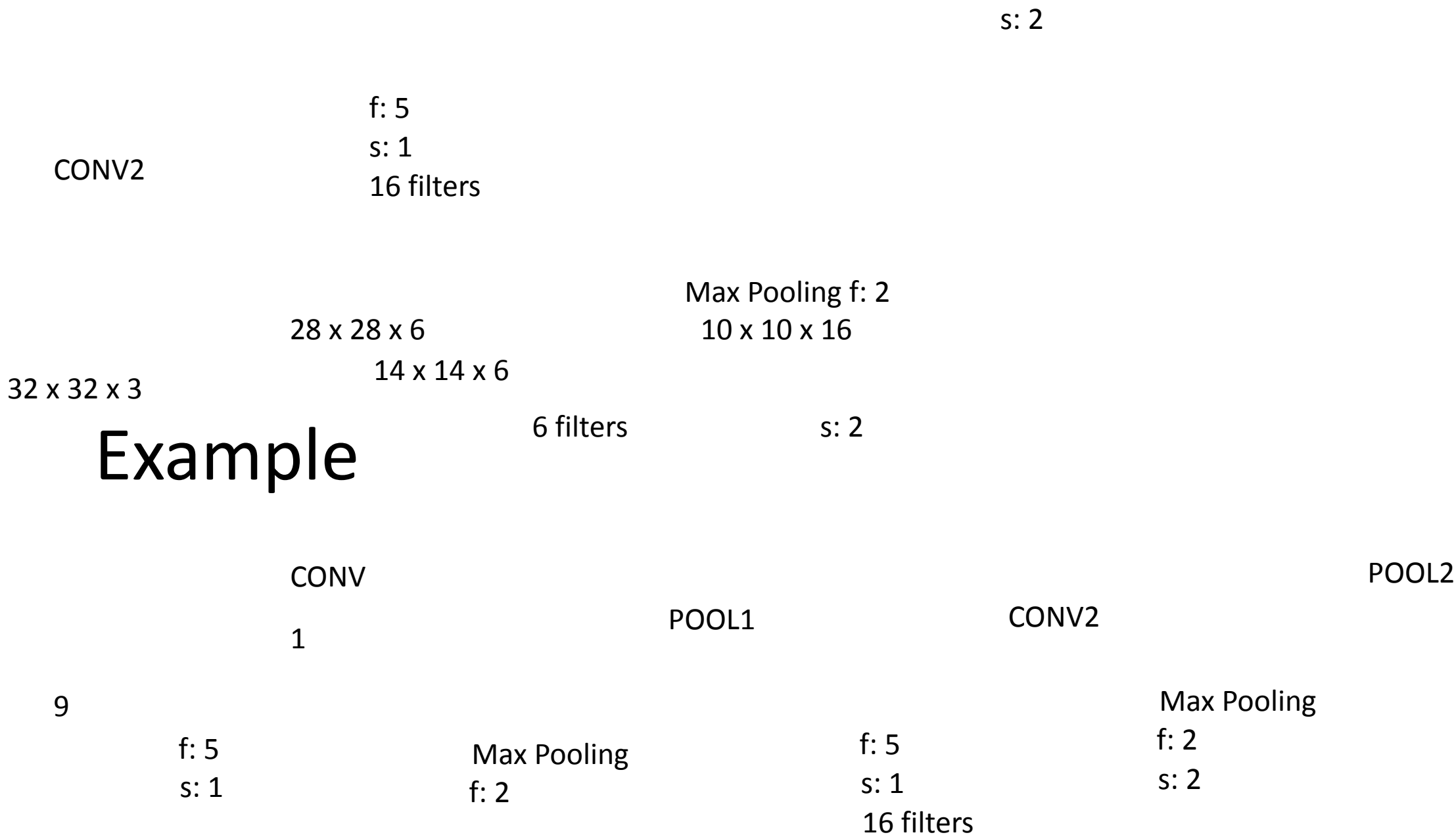
f: 2

s: 2

CONV1

POOL1

9



Example

32 x 32 x 3

28 x 28 x 6

14 x 14 x 6

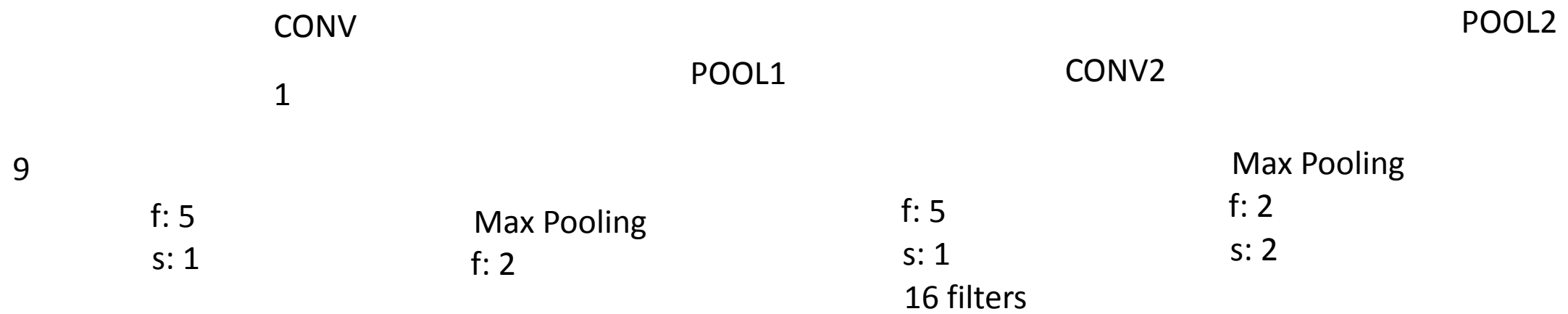
10 x 10 x 16

5 x 5 x 16

6 filters

s: 2

Example



32 x 32 x 3 400 x 1

28 x 28 x 6
14 x 14 x 6
10 x 10 x 16
5 x 5 x 16

Example⁹

CONV
1

f: 5
s: 1
6 filters

POOL1
Max Pooling
f: 2
s: 2

CONV2

f: 5
s: 1
16 filters

POOL2

Max Pooling
f: 2
s: 2

32 x 32 x 3 400 x 1

28 x 28 x 6

FC3

120 unites
14 x 14 x 6
10 x 10 x 16
5 x 5 x 16

Example

6 filters

s: 2

CONV

1

POOL1

CONV2

POOL2

9

f: 5
s: 1

Max Pooling
f: 2

f: 5
s: 1
16 filters

Max Pooling
f: 2
s: 2

32 x 32 x 3 400 x 1

28 x 28 x 6

FC3

120 unites
14 x 14 x 6

Will have a weight matrix of size 120 x 400

10 x 10 x 16

5 x 5 x 16

Example

6 filters

s: 2

CONV

1

POOL1

CONV2

POOL2

9

f: 5

s: 1

Max Pooling

f: 2

f: 5

s: 1

16 filters

Max Pooling

f: 2

s: 2

28 x 28 x 6 FC3

32 x 32 x 3

FC4

14 x 14 x 6
10 x 10 x 16
5 x 5 x 16

120 unites 84
unites

400 x 1

Example

CONV

1

f: 5
s: 1
6 filters

Max Pooling
f: 2
s: 2

CONV2

9

POOL1

f: 5

s: 1
16 filters

s: 2

Max Pooling
f: 2

POOL2

14 x 14 x 6
10 x 10 x 16
5 x 5 x 16

32 x 32 x 3

28 x 28 x 6 FC3

FC4

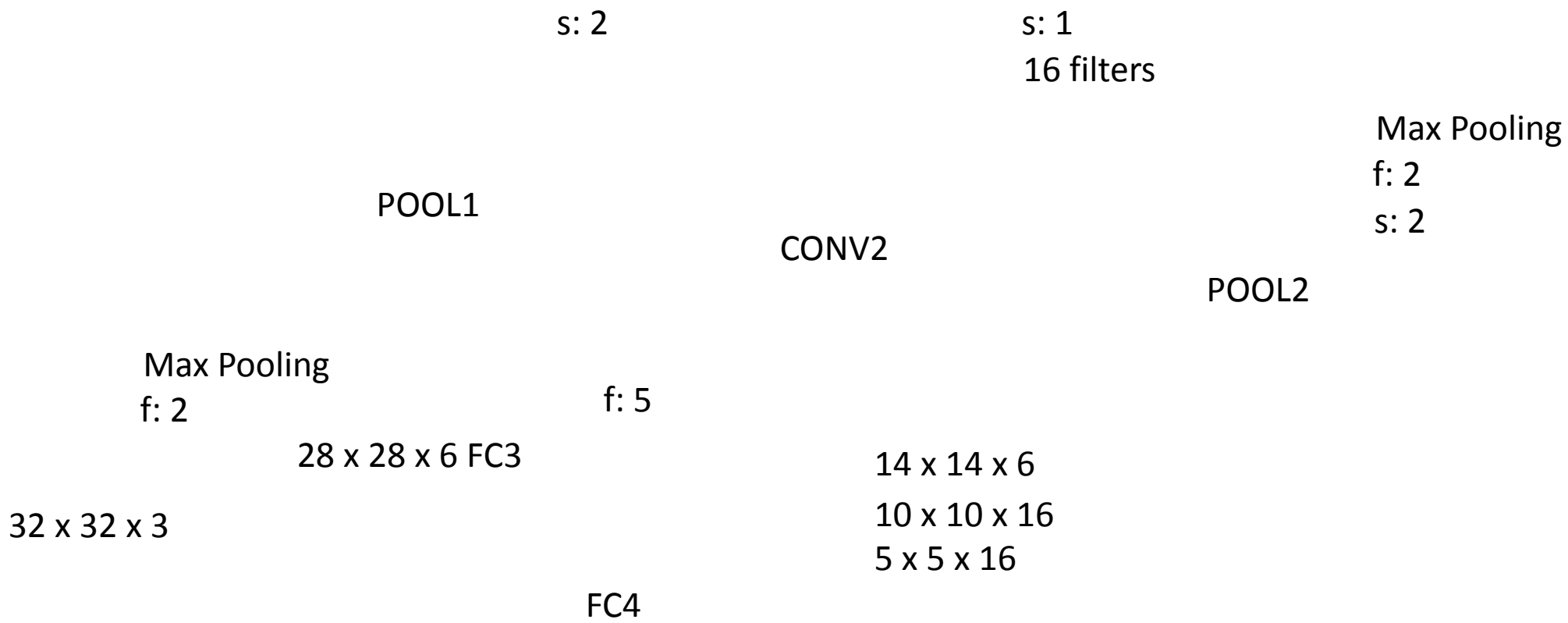
120 unites 84
unites
Softmax Unit

400 x 1

Example

CONV
1

f: 5
s: 1
6 filters



120 units 84 units

Softmax Unit

Will have 10 outputs, one
for each digit

400 x 1

CNNs

- As you saw in the example, there could be a large number of hyperparameters
- A common practice is to look for their values in literature, and use what has worked for others

CNNs

- Another thing to remember is that
- As we go deeper in to a CNN, width and height of input usually decreases,
- whereas the number of channels increases